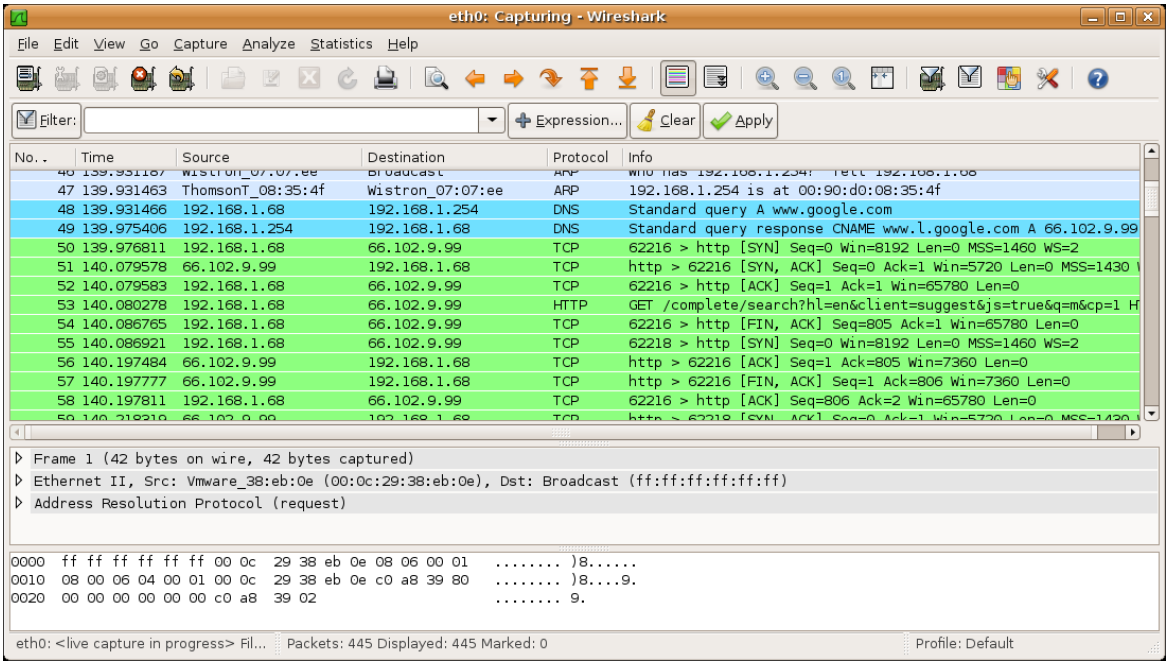
ARQUITECTURA DE REDES I / ARQUITECTURA DE REDES



GP23 Práctica sobre el protocolo HTTP

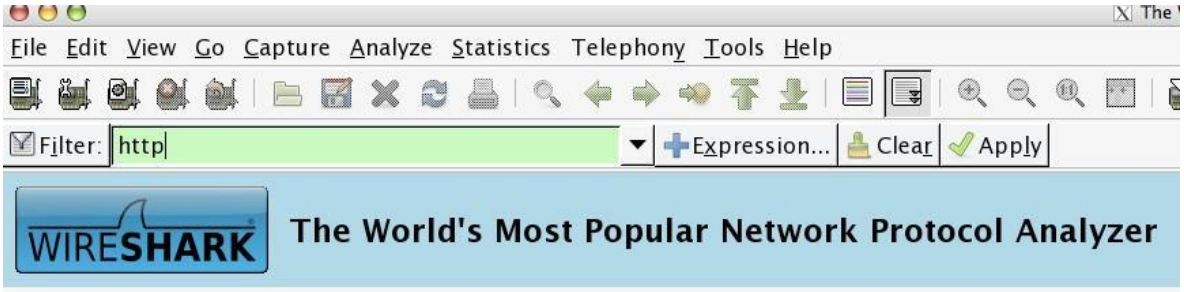
El objetivo de esta tarea es observar el funcionamiento de HTTP en diferentes escenarios.

Interacciones HTTP Básicas

Para comenzar a analizar el funcionamiento de HTTP vamos a comprobar cómo descarga el navegador una página web sencilla que tan sólo constará de un documento HTML de pequeño tamaño sin objetos incrustados (e.g., imágenes, vídeos, ...). También emplearemos el analizador de red Wireshark que ya conoce de sesiones anteriores.

Para ellos vamos a seguir los siguientes pasos:

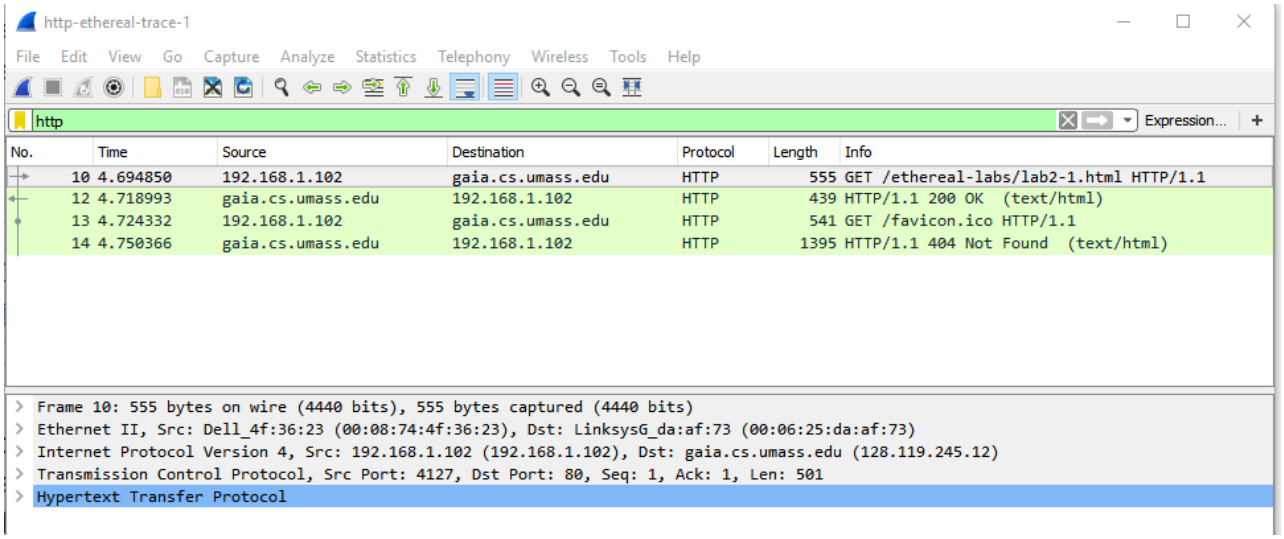
1. Arranque un navegador web (Internet Explorer, Mozilla Firefox, Google Chrome…).
2. Arranque el analizador de red Wireshark (desde un terminal ejecute: ***sudo wireshark***) (si se ejecuta sin *sudo* no es posible la captura de paquetes). Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena '**http**' (sin las comillas) en la ventana de especificación de filtros de visualización (*display-filter*). Si no hiciéramos esto veríamos todo el tráfico de red que es capaz de capturar nuestra tarjeta.



3. Espere aproximadamente un minuto (ya veremos por qué), y comience a capturar paquetes empleando Wireshark (Menú *Capture -> Interfaces -> Start*).
4. Introduzca la siguiente dirección en la barra de direcciones de su navegador:
   http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
   Su navegador mostrará una página que sólo constará de una línea de texto.

5. Detenga la captura de paquetes en Wireshark (Menú *Capture -> Interfaces -> Stop*).

La ventana de Wireshark debería mostrar un aspecto similar a la que se muestra en la siguiente figura. En el caso de no haber podido realizar la captura adecuadamente, pida al profesor que se la facilite (*http-ethereal-trace-1*).



El ejemplo de la figura anterior muestra una ventana con un listado de paquetes que contiene los dos mensajes HTTP capturados: el mensaje GET (desde su navegador hacia el servidor web gaia.cs.umass.edu) y el mensaje de respuesta desde el servidor hacia su navegador. Justo debajo de la ventana que muestra el listado de mensajes encontrará una ventana que muestra el contenido del paquete que esté seleccionado (el que aparece resaltado en el listado anterior). Dado que los mensajes HTTP se encapsulan en segmentos TCP que a su vez van dentro de datagramas IP que son transportados en una trama (en este caso Ethernet), Wireshark muestra también la información de las cabeceras de la trama, del datagrama y del segmento. Como en este caso tan sólo estamos interesados en HTTP ocultaremos toda esa información. Para ello compruebe que al lado de la información de trama, Ethernet, IP y TCP aparece un signo '>' (que significa que hay información oculta que no es mostrada). Compruebe también que precediendo la línea de HTTP aparece un signo 'v', o clique en la flecha > para mostrar la información.

Analice la información de los mensajes HTTP de petición y respuesta y conteste las preguntas que se plantean. En la respuesta a las preguntas debe indicar las porciones de los mensajes de petición y respuesta en las que basa su contestación:

1. ¿Qué versión de HTTP emplea su navegador? ¿Qué versión de HTTP ejecuta el servidor?

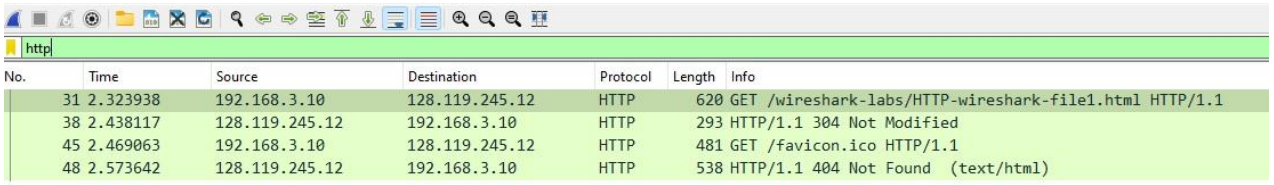
El navegador y el servidor usan HTTP 1.1.

2. ¿Qué idiomas indica su navegador al servidor que está dispuesto aceptar en la respuesta? ¿Qué relación tiene esta información con el lenguaje que emplea en su navegador?

Los idiomas indicados por el servidor son español e inglés que son los mismos idiomas que usa el navegador usado.

3. ¿Cuál es la dirección IP de su ordenador? ¿Y del servidor web al que está accediendo?

La dirección del ordenador es **192.168.3.10**, mientras que la dirección del servidor web al que accedemos es **128.119.245.12**



4. ¿Cuál es el código de estado devuelto a su navegador por el servidor? ¿Cuál es el significado de ese código de estado?

Al hacer el ejercicio en clase, el navegador nos devuelve un 200 OK como código, cuyo significado es que el contenido se ha encontrado y se va a enviar de manera correcta al cliente.
Al volver a hacer las capturas en casa, al estar la dirección ya en la caché del dispositivo, nos devuelve un 304 Not Modified, que como su nombre indica nos dice que la web no se ha modificado y evita el reenviar la información de nuevo al cliente.

5. ¿Cuándo fue modificado por última vez el fichero HTML que el servidor le está devolviendo?

La fecha que aparece como last modified es el viernes 21 de Octubre del 2022 a las 05:59:02 horas

```
If-Modified-Since: Fri, 21 Oct 2022 05:59:02 GMT\r\n
```

En esta captura aparece "If-Modified-Since" ya que la dirección del servidor ya se encuentra almacenada en el dispositivo.

6. ¿Cuántos bytes de contenido está devolviendo el servidor al navegador? ¿Coincide con el tamaño de la trama? Razone su respuesta.

El contenido que devuelve el servidor es de 540 bytes, que no coincide con la trama que presenta un tamaño de 128 bytes.

7. Inspeccionando la secuencia de octetos que representan el contenido de la trama, que aparece en la parte inferior de la ventana, ¿observa alguna cabecera que no se muestre en la ventana del listado de protocolos del paquete (la ventana superior)? Si es así, enuncie cuál o cuáles son. Para ver esto vaya seleccionando los distintos protocolos de la trama y observe qué octetos corresponden a la cabecera de dicho protocolo.

No hay ningún cambio entre ambas ventanas de visualización, ya que ambas muestran la misma información al usuario.

En la respuesta a la cuestión número 5, puede que le sorprenda descubrir que el documento que acaba de descargar ha sido modificado un minuto antes de que lo descargara. En este caso particular es debido a que el servidor actualiza la fecha de modificación del fichero una vez por minuto. De este modo, si espera un minuto entre accesos, parecerá que el fichero ha sido modificado recientemente y, por tanto (ya veremos por qué), su navegador descargará una 'nueva' copia del documento.

Interacciones HTTP condicionales

Tal y como hemos visto, la mayoría de los navegadores web guardan una copia en caché de parte de los archivos a los que acceden lo que posibilita la realización de peticiones GET condicionales cuando tratamos de obtener un objeto http. En este apartado vamos a comprobar el funcionamiento de este mecanismo. Para poder apreciarlo adecuadamente es necesario que nos aseguremos que la caché esté vacía. Para hacer esto en Firefox, selecciona *Historial-> Limpiar el historial reciente*. En Internet Explorer, deberás seleccionar *Seguridad->Eliminar el historial de exploración*. Estas acciones borrarán el contenido la caché de su navegador. A continuación, haga lo siguiente:

1. Arranque el navegador y cerciórese de que la caché de su navegador esté vacía, tal y como hemos explicado en el apartado anterior.
2. Arranque el analizador de red Wireshark y comience a capturar paquetes.
3. Introduzca la siguiente URL en la barra de direcciones de su navegador: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html

En la ventana del navegador verá un fichero HTML sencillo de tan sólo cinco líneas.

1. Acto seguido, actualice la página en el navegador (la actualización ha de ser rápida).
2. Detenga la captura de paquetes en Wireshark e introduzca 'http' en la ventana de especificación de filtros de visualización de modo que tan sólo se muestren los mensajes HTTP en la ventana de listado de paquetes.

Responda a las siguientes preguntas:

1. Inspeccione los contenidos de la primera petición GET de su navegador hacia el servidor. ¿Observa alguna linea *'If-Modified-Since'* en la petición GET?. Indique el contenido de dicha línea.

No existe tal línea en la primera petición, ya que al no estar la información en la caché del dispositivo, no puede conocer la última fecha de modificación previamente a la respuesta del servidor.

```
Hypertext Transfer Protocol
  GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n
    [Expert Info (Chat/Sequence): GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
      [GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n]
      [Severity level: Chat]
      [Group: Sequence]
    Request Method: GET
    Request URI: /wireshark-labs/HTTP-wireshark-file2.html
    Request Version: HTTP/1.1
  Host: gaia.cs.umass.edu\r\n
  Connection: keep-alive\r\n
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/106.0.0.0 Safari/537.36\r\n
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9\r\n
  Accept-Encoding: gzip, deflate\r\n
  Accept-Language: es-ES,es;q=0.9,en;q=0.8\r\n
  \r\n
  [Full request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
  [HTTP request 1/1]
  [Response in frame: 14747]
```

2. Analice el contenido de la respuesta del servidor. ¿Cuál es la fecha de última modificación del fichero HTML en la respuesta del servidor a la primera petición GET?

La fecha que aparece como Last Modified es el viernes 21 de octubre de 2022 a las 05:59:02 horas.

3. ¿Contiene dicha respuesta el fichero HTML que se pedía?

Si contiene la respuesta el fichero HTML, indicado por el código 200 OK.

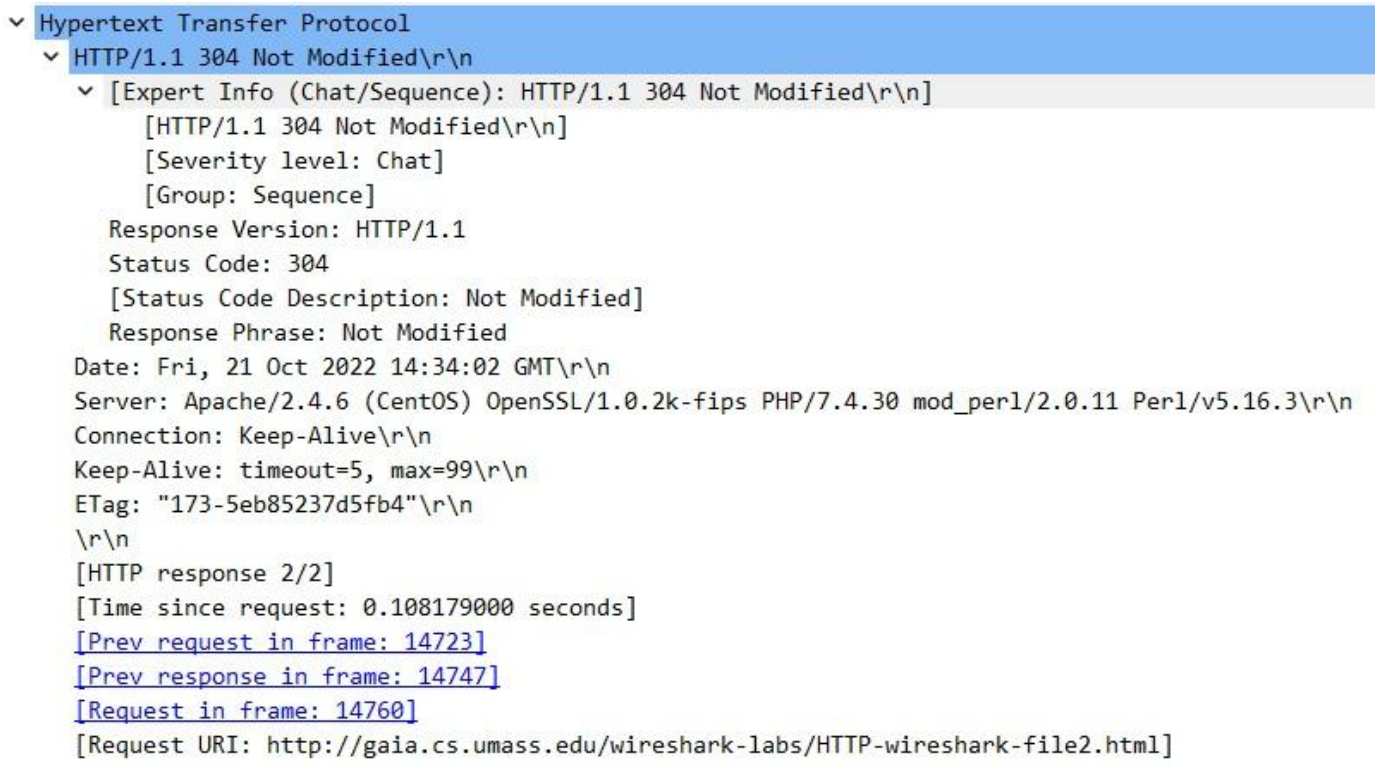
```
Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
    [Expert Info (Chat/Sequence): HTTP/1.1 200 OK\r\n]
      [HTTP/1.1 200 OK\r\n]
      [Severity level: Chat]
      [Group: Sequence]
    Response Version: HTTP/1.1
    Status Code: 200
    [Status Code Description: OK]
    Response Phrase: OK
  Date: Fri, 21 Oct 2022 14:33:59 GMT\r\n
  Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.30 mod_perl/2.0.11 Perl/v5.16.3\r\n
  Last-Modified: Fri, 21 Oct 2022 05:59:02 GMT\r\n
  ETag: "173-5eb85237d5fb4"\r\n
  Accept-Ranges: bytes\r\n
  Content-Length: 371\r\n
  Keep-Alive: timeout=5, max=100\r\n
  Connection: Keep-Alive\r\n
  Content-Type: text/html; charset=UTF-8\r\n
  \r\n
  [HTTP response 1/2]
  [Time since request: 0.116437000 seconds]
  [Request in frame: 14723]
  [Next request in frame: 14760]
  [Next response in frame: 14761]
  [Request URI: http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html]
  File Data: 371 bytes
Line-based text data: text/html (10 lines)
```

4. Ahora analice el contenido de la segunda petición GET desde su navegador al servidor. ¿Ve alguna línea *'If-Modified-Since'* en dicha petición? Si es así, ¿qué información aparece a continuación de dicha cabecera?

Si aparece, ya que la caché del dispositivo conoce la última fecha en la que se modificó y puede meterla en el mensaje, siendo la información que aparece "*If-Modified-Since: Fri, 21 Oct 2022 05:59:02*"

5. ¿Cuál es el código de estado HTTP y el mensaje devuelto por el servidor en respuesta a este segundo GET? ¿Devuelve el servidor explícitamente el contenido del fichero? Explique qué ha sucedido.

El código de estado es 304, es decir, que no se ha modificado la web que tenemos almacenada en la caché y no tenemos que volver a bajarla. Por tanto, el servidor no devuelve de manera explícita el contenido que le hemos pedido.



Descarga de documentos grandes

En todos los ejemplos hasta ahora, los documentos que hemos descargado eran documentos HTML sencillos. Comprobemos qué sucede si descargamos un fichero HTML grande (sin objetos dentro del HTML). Haga lo siguiente:

1. Arranque su navegador web.
2. Arranque el analizador de paquetes Wireshark y comience a capturar paquetes.
3. Introduzca la siguiente URL en al navegador:

   http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html

   El navegador debería mostrar un documento HTML de cierta longitud.
4. Detenga la captura de paquetes en Wireshark e introduzca 'http' en la ventana de especificación de filtros de visualización de manera que tan sólo se muestren los mensajes HTTP.

En la ventana de listado de paquetes debería ver un mensaje http GET seguido de una respuesta multi-paquete a este mensaje. Esta respuesta multipaquete merece una explicación. Hemos estudiado que el mensaje de respuesta HTTP consta de una línea de estado seguido de líneas conteniendo distintas cabeceras seguidas de una línea en blanco, seguida del cuerpo de la entidad. En el caso de GET, este cuerpo es el fichero HTML pedido. En el caso en que nos encontramos, este fichero HTML es bastante grande, los 4500 bytes de los que consta no caben en un único mensaje TCP. Esto obliga a que la respuesta HTTP se divida en varias partes, viajando cada una de ellas en un segmento TCP separado (recomendamos consultar el libro para ver esto). Dependiendo de la configuración de preferencias, Wireshark registra cada segmento TCP como un paquete independiente o los reensambla, mostrándolos juntos. Para mostrarlos reensamblados cambie las preferencias en Wireshark de la forma siguiente: ***Edit->Preferences, Protocols, TCP*** activar la casilla "Allow subdissector to reassemble TCP streams".

El hecho de que una única respuesta HTTP se divida en múltiples paquetes TCP se indica por parte de Wireshark mediante el mensaje "Continuation" en cada uno de los segmentos TCP que siguen al primero. Esto es únicamente un mecanismo que emplea Wireshark para indicar esto, no existe un mensaje "*Continuation*" ni en HTTP ni en TCP.

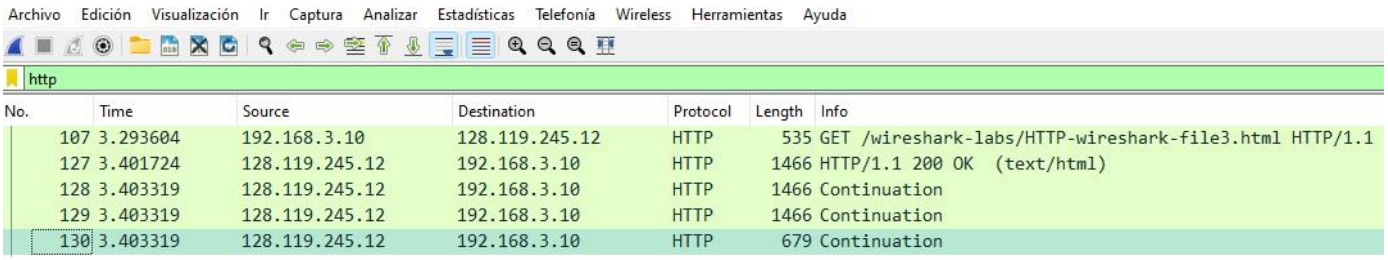
Para que se vean de forma independiente los segmentos TCP, hay que cambiar las preferencias en Wireshark: en ***Edit->Preferences, Protocols, TCP*** hay que desactivar la casilla "Allow subdissector to reassemble TCP streams". Responda las siguientes cuestiones:

1. ¿Cuántos mensajes de petición GET han sido enviados por el navegador?

Solo se ha enviado un mensaje con la petición GET al servidor.

2. ¿Cuántos segmentos TCP con datos han sido necesarios para una única respuesta HTTP?

Se han necesitado 4 segmentos TCP para la respuesta.



3. ¿Cuál es el código de estado y el mensaje asociado con la respuesta a la petición GET?

El código de estado es 200 OK, por lo que se envía el contenido de la web al cliente.

Documentos HTML con Objetos Empotrados

Una vez que hemos visto la forma en que Wireshark muestra el tráfico de paquetes capturados para mensajes HTML grandes, vamos a analizar qué sucede cuando nuestro navegador descarga un fichero con objetos empotrados, por ejemplo un fichero que incluye otros objetos (en el ejemplo que veremos, imágenes) que se almacenan en otro u otros servidores.

Lleve a cabo las siguientes acciones:

1. Inicie el navegador web y asegúrese que ha vaciado la caché tal y como hemos explicado anteriormente.
2. Arranque Wireshark y comience a capturar paquetes.
3. Introduzca la siguiente URL en el navegador:

http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html

El navegador debería mostrar un fichero HTML con dos imágenes. Estas dos imágenes se referencian en el fichero HTML, es decir, las dos imágenes no están contenidas en el HTML sino que lo que aparece en el HTML son las URLs donde se ubican las imágenes. Por ejemplo, la primera imagen aparece referenciada del siguiente modo empleando la etiqueta html *img*. Puede acceder al código fuente de la página web cargada pinchando con el botón derecho del ratón en cualquier parte del contenido y, a continuación, "Ver código fuente de la página" o directamente pulsando la combinación Ctrl+U.

```
<img src="http://gaia.cs.umass.edu/pearson.png" WIDTH="140" HEIGHT="82" >
```

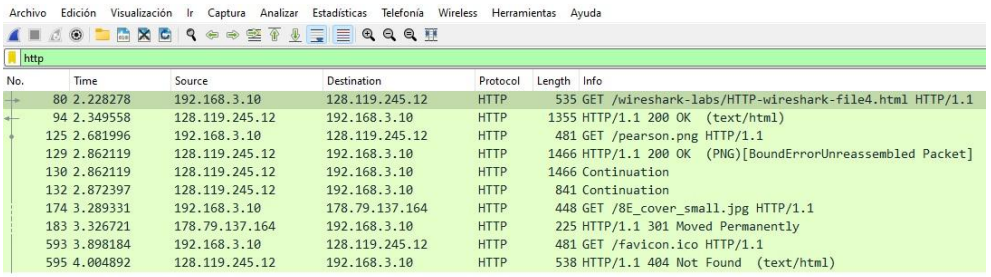
Si copia la URL que aparece a continuación de "src" y la pega en la barra de direcciones del navegador, comprobará cómo el navegador muestra una página que tan sólo contiene la primera imagen de las dos que aparecen en la página (hágalo después de contestar a las dos preguntas de este apartado). Tal y como hemos visto, el navegador tendrá que descargar esas imágenes de los sitios web indicados en el HTML.

4. Detenga la captura de paquetes de Wireshark e introduzca 'http' en la ventana de especificación de filtros de visualización de modo que sólo se muestren los mensajes HTTP.

Responda las siguientes cuestiones:

1. ¿Cuántos mensajes GET ha enviado el navegador? ¿A qué direcciones se han enviado esos mensajes?

El navegador ha enviado 4 mensajes con GET, 3 de ellos al **128.119.245.12** y uno al **178.79.137.164**



2. ¿Puede determinar si las imágenes han sido descargadas sucesivamente (se pide una imagen, se descarga y después se pide la otra) o en paralelo (se lanzan las dos descargas de forma simultánea)? Razone su respuesta.

Al observar el tiempo de captura en wireshark, podemos observar como las capturas correspondientes a la descarga de la imagen son el 129, 130 y 132, ya que el jpg no se descarga (código de estado 301).
Viendo los correspondientes tiempos de captura, es decir, 2.862119 para los dos primeros y 2.872397 para la última parte de la imagen, podemos concluir que la descarga se ha realizado en paralelo con otra orden intercalada entre las descargas.

```
131 2.862159    192.168.3.10    128.119.245.12    TCP    54 52588 → 80 [ACK] Seq=909 Ack=4126 Win=131072 Len=0
```

Autenticación HTTP

Finalmente vamos a tratar de visitar un sitio web protegido mediante una clave y vamos a examinar la secuencia de mensajes http intercambiados para dicho sitio.

La URL http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html está protegida mediante una clave. El nombre de usuario es 'wireshark-students' (sin las comillas) y la clave es 'network' (sin las comillas una vez más). Vamos a acceder a este sitio 'protegido' mediante esta pareja usuario/contraseña. Haga lo siguiente:

1. Asegúrese que la caché del navegador está vacía, tal y como hemos explicado anteriormente, y cierre el navegador. A continuación, arranque el navegador de nuevo.
2. Arranque Wireshark y comience a capturar paquetes
3. Introduzca la siguiente URL en el navegador:

   http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html

   Introduzca el nombre de usuario y la contraseña que comentábamos antes cuando se le pida.
4. Detenga la captura de paquetes de Wireshark e introduzca 'http' en la ventana de especificación de filtros de visualización de modo que sólo se muestren los mensajes HTTP.

Examine la salida de Wireshark. Se recomienda leer primero el siguiente documento sobre autenticación HTTP: http://frontier.userland.com/stories/storyReader$2159

Responda a las siguientes cuestiones:

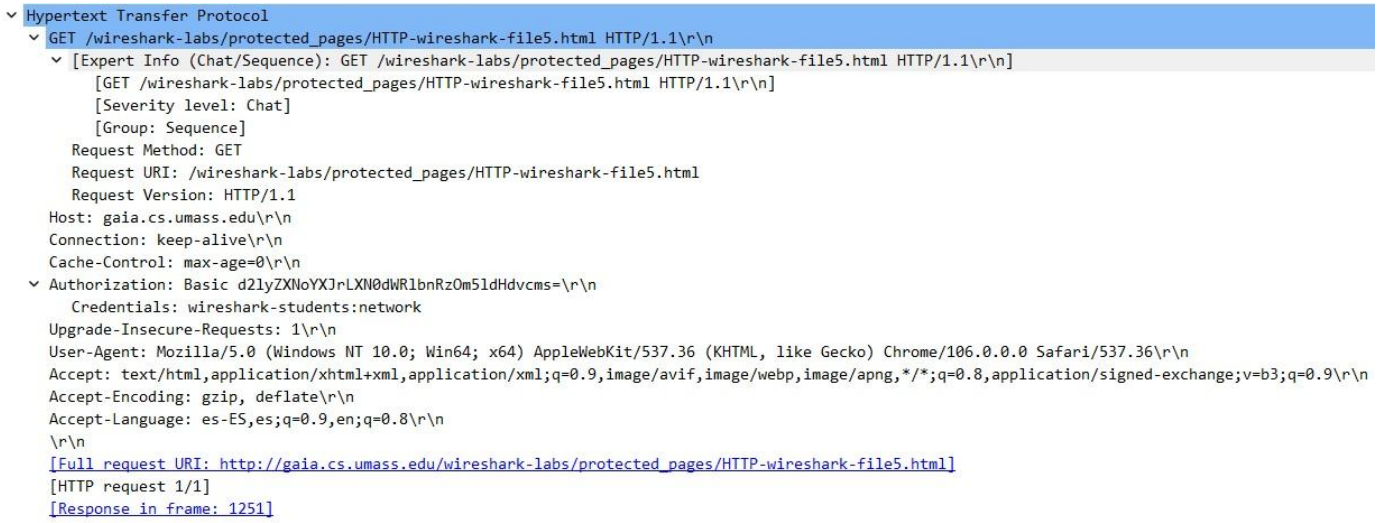
1. ¿Cuál es el código de respuesta y el mensaje del servidor en respuesta al mensaje GET inicial de su navegador?

El código de respuesta que nos manda el servidor en el mensaje inicial es **401 Unauthorized**, con el correspondiente mensaje de **Unauthorized**.

```
▾ Hypertext Transfer Protocol
  ▾ HTTP/1.1 401 Unauthorized\r\n
    ▾ [Expert Info (Chat/Sequence): HTTP/1.1 401 Unauthorized\r\n]
        [HTTP/1.1 401 Unauthorized\r\n]
        [Severity level: Chat]
        [Group: Sequence]
      Response Version: HTTP/1.1
      Status Code: 401
      [Status Code Description: Unauthorized]
      Response Phrase: Unauthorized
    Date: Sat, 22 Oct 2022 13:30:33 GMT\r\n
    Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.30 mod_perl/2.0.11 Perl/v5.16.3\r\n
    WWW-Authenticate: Basic realm="wireshark-students only"\r\n
  ▾ Content-Length: 381\r\n
      [Content length: 381]
    Keep-Alive: timeout=5, max=100\r\n
    Connection: Keep-Alive\r\n
    Content-Type: text/html; charset=iso-8859-1\r\n
    \r\n
    [HTTP response 1/1]
    [Time since request: 0.108015000 seconds]
    [Request in frame: 70]
    [Request URI: http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html]
    File Data: 381 bytes
▸ Line-based text data: text/html (12 lines)
```

2. Cuando su navegador envía el mensaje HTTP GET por segunda vez, ¿qué nuevo campo se incluye en el mensaje GET?

Incluye un nuevo campo que incluye el nombre de usuario y la contraseña del mismo.



El nombre de usuario (wireshark-students) y la contraseña (network) introducidas se codifican en la cadena de caracteres (d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=) que aparece a continuación de la cabecera *"Authorization: Basic"* en el mensaje GET del cliente. Aunque pudiera parecer que se envían cifrados, simplemente están codificados en un formato conocido como Base64. Puede comprobar esto accediendo a cualquier codificador de Base64 web, donde se vería que se concatenan usuario y contraseña codificados en Base64. Wireshark también incluye un decodificador, expandiendo la información de la cabecera nos muestra el valor decodificado.

Cualquier persona con acceso a una utilidad como Wireshark podría capturar paquetes de la red y trasladarlos de Base64 a ASCII. Esto debería dejar claro que el empleo de sistemas sencillos de autenticación en HTTP como los descritos aquí no son seguros a menos que se tomen medidas adicionales. La solución habitualmente usada para esto es cifrar las comunicaciones mediante web seguro https, que implica añadir una capa con el protocolo SSL o TLS entre las capas de aplicación (protocolo HTTP) y transporte (protocolo TCP).